

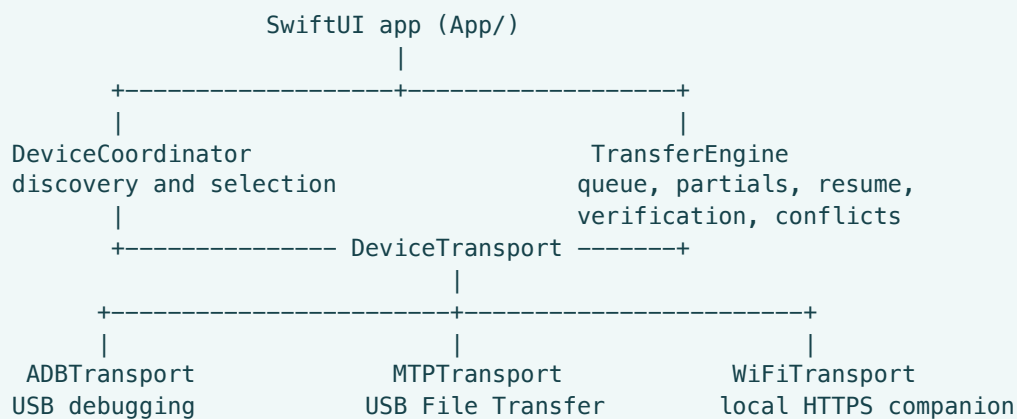
Porter

Developer guide for a native macOS file-transfer app and its paired Android companion.

Porter makes an Android phone behave like a drive over USB or a trusted local network connection. The Swift package owns transfer semantics; the macOS app owns presentation and saved pairings; the Android companion supplies the local Wi-Fi endpoint.

Current boundary. ADB and MTP are implemented USB paths. The paired Wi-Fi client and Android companion compile and implement the complete v1 protocol. The next proof point is installation and end-to-end validation on real Android hardware.

1. System shape



`Kit/` contains the UI-independent model, transport, discovery, and transfer engine. The boundary is intentional: transfer failure behavior can be tested without a cable, a device lock screen, or a network disconnect occurring at an inconvenient moment.

Transport contract

Every transport implements `DeviceTransport` and publishes `TransportCapabilities`. The engine reads capabilities rather than checking a transport kind, so differences such as resumability and device hashing are explicit data rather than scattered conditional logic.

Transport	Connection	What it provides	Limit to expose
ADB	USB debugging	Ranged reads, resumable writes, SHA-256 on device, mtime, reliable size and capacity.	Requires developer options and USB debugging.
MTP	USB File Transfer	Browse and copy without USB debugging.	Serial operations; no dependable resume or device checksum; mtime and capacity are best effort.
Wi-Fi	Paired Android companion on the same LAN	Range downloads, offset uploads, SHA-256/MD5, mtime, and capacity.	Requires the companion and all-files access. The Mac does not browse mDNS advertisements yet.

Transfer guarantees

Incoming bytes always land in a `.porterpart` sidecar. A final name appears only when the transfer meets that transport's completion rules. ADB and Wi-Fi resume interruptions; MTP restarts them because Android MTP implementations do not provide trustworthy range behavior. ADB and Wi-Fi compare a Mac hash with a device hash. MTP is marked *unverified* because it has no checksum operation.

2. USB paths

ADB

ADB is preferred when a device exposes both USB modes. The engine can use a native bulk pull for a fresh copy and a streamed range read for a resumed copy. It preserves modification dates and asks the device to hash completed bytes before reporting verified success. Small files at or below 8 MiB can share an ADB pull and hashing invocation to reduce per-process overhead.

MTP

MTP is the direct fallback for a phone in ordinary File Transfer mode. Porter implements the protocol framing, USB bulk pipe, session layer, handle-to-path resolution, and transport directly. Session transactions are serialised: a response cannot be picked up by a later UI request after a fast folder change.

macOS ownership rule. `ptpcamerad` may claim an MTP interface before Porter. Porter starts an interface claimer and races at attach time. Start Porter before plugging in the phone; if another process wins, unplug and reconnect. `porterctl mtp` and `porterctl mtp-watch` diagnose the interface holder. Photos and Image Capture cannot use a phone while Porter holds its MTP interface.

3. Paired Wi-Fi

The Wi-Fi path is local-only: it has no account system, relay service, or Internet API. The companion starts an HTTPS server on port 53317 and advertises `_porter._tcp`. The current Mac UI intentionally asks for the phone's address, six-digit code, and displayed certificate fingerprint; Mac-side service browsing is a follow-up once the core pairing path is validated.

Pairing and trust sequence

1. The Android companion creates or reuses a self-signed RSA identity in Android Keystore and displays a six-digit pairing code plus the certificate SHA-256 fingerprint.
2. Porter posts the code and a client name to `POST /v1/pair` over TLS.
3. The Mac accepts the self-signed certificate for this request only, calculates the leaf certificate SHA-256 fingerprint, and compares it to both the fingerprint transcribed from the phone and the value in the pairing response.
4. Only matching values are saved. The bearer token goes in login Keychain; preferences contain only device name, host, port, and pinned fingerprint.
5. Later requests carry the bearer token and use a URL session that rejects any certificate other than the pinned fingerprint.

The companion stores hashes of issued tokens and rate-limits repeated bad pairing codes. It is not enough for a peer to know an address or intercept a code: the certificate presented at pairing must be the fingerprint read from the phone.

Android storage boundary

The user grants all-files access before starting sharing. The server canonicalises every requested path, permits it only beneath Android shared-storage roots, and refuses to modify a volume root. This is a second boundary behind authentication: a paired Mac can manage shared files but cannot request arbitrary system or app-private paths.

v1 protocol

Endpoint	Method	Purpose
<code>/v1/pair</code>	POST	Exchanges the six-digit code for a token, device name, and certificate fingerprint.
<code>/v1/info</code> , <code>/volumes</code> , <code>/list</code> , <code>/stat</code> , <code>/free</code>	GET	Device and filesystem metadata.
<code>/v1/mkdir</code> , <code>/delete</code> , <code>/move</code> , <code>/touch</code>	POST	Directory and metadata mutation.
<code>/v1/read</code>	GET	Raw bytes; supports an open-ended HTTP <code>Range</code> for resume.
<code>/v1/write</code>	PUT	Raw bytes at an explicit <code>offset</code> ; a zero-length request trims a partial upload.
<code>/v1/checksum</code>	GET	SHA-256 or MD5 of a shared-storage file.

4. Discovery, paths, and preflight

`USBDeviceMonitor` watches IOKit and distinguishes a charge-only phone from one exposing ADB or MTP. `DeviceMerge` combines USB, ADB, and persisted wireless records into one list and chooses the best usable transport.

Android user storage is commonly a FUSE presentation layer. The ADB path resolves its backing filesystem where possible so a FAT32 4 GiB limit can be detected before transfer. MTP does not reliably provide that identity and must not guess. The Wi-Fi companion reports capacity from the actual storage root and rejects a write chunk that does not fit.

5. Repository layout

Kit/	Swift package: model, transports, engine, discovery
Sources/PorterKit/	
Transport/ADB/	ADB process transport
Transport/MTP/	Direct MTP protocol, pipe, and session
Transport/WiFi/	v1 protocol, pairing, certificate-pinned client
Sources/porterctl/	Read-only diagnostic CLI
Tests/	172 tests in 22 suites
App/	SwiftUI app, saved Wi-Fi registry and pairing sheet
Android/	Companion Android application and Gradle wrapper
Docs/	Architecture, status, testing notes, this guide
project.yml	XcodeGen input

6. Build and test

Mac package

```
cd Kit
swift test
```

The current suite contains 172 tests in 22 suites. It covers the model, parsing, transfer planner/engine, transport capabilities, MTP protocol/session behavior, and Wi-Fi client/pairing validation paths.

macOS app

```
xcodegen generate
xcodebuild -project Porter.xcodeproj -scheme Porter -configuration Debug build
```

The app target needs a signing team in `project.yml`. macOS 14+ and Xcode 26 are the project baseline.

Android companion

```
cd Android
./gradlew :app:assembleDebug
```

Use JDK 17, Android SDK platform 35, and Gradle 9.6 or newer. Configure the Android SDK through `ANDROID_HOME` or the untracked `Android/local.properties` on a development machine. The project uses platform APIs only; it does not depend on a third-party HTTP server or TLS library.

7. Hardware validation plan

1. Install the debug APK on an Android 11+ phone and grant all-files access.
2. Start sharing, confirm the displayed local address and code, and pair from the Mac app.
3. Browse every advertised volume; copy a mixed directory in both directions; confirm mtime and checksum status.
4. Interrupt a download and an upload, then retry. Confirm a `.porterpart` resumes rather than becoming a final file.
5. Enter a wrong code repeatedly, verify rate limiting, then pair correctly. Change or clear the Android app's certificate and verify that the Mac rejects the changed fingerprint until re-paired.
6. Run a sustained mixed-file transfer and record device model, Android release, storage type, network type, throughput, and failure details.

8. Next phases

Phase	Status	Purpose
Wi-Fi hardware validation	Next	Prove the compiled pairing and transfer path on real devices before expanding it.
Mac mDNS browser	Planned	Discover <code>_porter._tcp</code> services, while keeping the pairing code and fingerprint as the trust gate.
Finder File Provider	Not started	Expose a limited Finder surface through an XPC proxy to the main app. A sandboxed extension cannot spawn <code>adb</code> .
Release hardening	Planned	Signing, notarisation, bundled ADB, and broader MTP/Wi-Fi compatibility testing.
Phase 3	Not started	Photo import, watched folders, APK sideloading, and Quick Look.